

PROMPT PARTY

MASTER PRODUCT AND INTEGRATION SPECIFICATION

Standalone Twitch-ready audience-controlled AI party-game platform

Packet	00
Status	Claude Code build-ready specification
Date	July 27, 2026

Build principle: reliability first, audience agency second, chaos only through controlled game objects.

Source Basis

This packet combines three supplied foundations:

- The shared Prompt Party game-engine contract developed in the conversation.
- The supplied Entertainment System Specification covering all five games, including comedy rules, power-ups, chaos events, pacing, failure recovery, and clip moments.
- The uploaded visual-show HTML prototype, which already separates Overview, Lobby, Shared System, Broadcast, Audience, Producer, and individual game views.

Where a source mechanic cannot be executed reliably in version one, this packet preserves its intent and assigns a controlled implementation or explicit fallback rather than silently deleting it.

1. Product Definition

Prompt Party is a standalone live game-show application in which six to eight AI participants perform as contestants, actors, artists, lawyers, rappers, witnesses, or roasters while two configurable AI commentator seats host, react, referee, and sometimes judge. Human viewers submit approved challenges, trigger controlled modifiers, and help decide outcomes.

NON-NEGOTIABLE SEPARATION: Use Team Talk as a reference implementation and source of reusable patterns. Create a separate repository, service, port, configuration, database, storage path, generated-media directory, UI, and deployment. Do not overwrite or convert the serious Team Talk application.

2. Version-One Games

Module	Primary output	Audience role	Commentator role
Art Showdown	Generated images	Prompt, modifier, vote	Expectation banter, visual critique, scoring
Rap Battle	Written verses; optional audio	Topic, forced words, vote	Hype, critique, scoring
AI Court	Arguments, testimony, verdict	Case facts, evidence, jury vote	Host, legal color, referee
AI Improv	Short scenes	Suggestions, twists, best-moment vote	Host, rule enforcement, recap
Roast Battle	Playful roast exchanges	Approved target/style, vote	Color commentary, safety referee, scoring

3. Product Surfaces

Surface	Purpose	Must show	Must never show
Broadcast	OBS/Twitch presentation	Challenge, cast, phase, output, captions, voting, result	API keys, raw exceptions, private moderation, hidden prompts
Audience	Room-code browser experience	Join, submit, choose, vote, confirmation	Producer controls or unapproved content
Producer	Private control room	Approvals, phase controls, retries, overrides, provider health	Nothing is hidden from producer except secrets masked by default
Replay	Round record and exports	Prompt, outputs, comments, scores, votes, failures, recoveries	Secrets and disallowed raw provider data

The uploaded visual prototype already reflects these separate views and should be treated as the visual-direction reference, not as the final application architecture.

4. Shared 12-Phase Lifecycle

Phase	Function
LOBBY	Cast, room, provider health, game selection
SUBMISSION_OPEN	Audience submits approved input types
SUBMISSION_REVIEW	Automated screening and producer review
PROMPT_LOCKED	Final challenge becomes immutable for the round
ROUND_INTRO	Broadcast framing and contestant assignment
CONTESTANT_PLANNING	Short structured plans or opening positions
CREATION_ACTIVE	Generation, performance, testimony, or scene play

Phase	Function
PRE_REVEAL_COMMENTARY	Commentators cover waiting or deliberate
REVEAL	Generated work or completed performance is shown
FINAL_JUDGING	Grounded critique and structured scores
AUDIENCE_VOTING	Audience vote opens and closes
SCORING / WINNER / POST-ROUND	Calculate, announce, banter, persist, reset
AUTHORITY RULE: Only the producer or trusted controller advances official state. A model response, audience vote, media callback, or provider completion cannot advance the round by itself unless the current state explicitly permits an automatic completion transition.	

5. Core Data Contracts

5.1 Participant

```
{
  "participant_id": "judge_01",
  "display_name": "Verdict",
  "provider": "openai_compatible",
  "model": "configured-model",
  "seat_type": "commentator",
  "capabilities": [
    "text_generation",
    "vision_input",
    "structured_output"
  ],
  "persona_id": "brutal_critic",
  "avatar": "/assets/avatars/verdict.webp",
  "enabled": true,
  "timeout_seconds": 45,
  "max_retries": 1
}
```

5.2 Event envelope

```
{
  "event_id": "evt_01J...",
  "event_type": "judge.commentary_created",
  "show_id": "show_01J...",
  "round_id": "round_01J...",
  "game_id": "art_showdown",
  "phase": "PRE_REVEAL_COMMENTARY",
}
```

```

"actor_type": "ai",
"actor_id": "judge_01",
"timestamp": "2026-07-27T17:00:00Z",
"public": true,
"payload": {}
}

```

5.3 Controlled modifier

```

{
  "modifier_id": "genre_shift",
  "game_id": "improv",
  "display_name": "Genre Shift",
  "allowed_phases": [
    "CREATION_ACTIVE"
  ],
  "target_types": [
    "scene"
  ],
  "duration": "next_two_turns",
  "requires_producer_approval": true,
  "audience_selectable": true,
  "effect": {
    "type": "scene_constraint",
    "instruction": "{{approved_genre}}"
  },
  "fallback": {
    "on_invalid_phase": "reject"
  }
}

```

6. Game Module Interface

```

class GameModule(Protocol):
    manifest: GameManifest
    def allowed_actions(self, state: RoundState) -> list[str]: ...
    def validate_transition(self, old: str, new: str) -> None: ...
    def build_ai_request(self, action: GameAction, context: GameContext) -> AIRequest: ...
    def parse_ai_response(self, action: GameAction, raw: str) -> StructuredOutput: ...
    def apply_modifier(self, modifier: Modifier, state: RoundState) -> RoundState: ...
    def calculate_score(self, round_record: RoundRecord) -> ScoreResult: ...
    def public_state(self, round_record: RoundRecord) -> dict: ...

```

Manifest field	Required meaning
game_id	Stable module identifier
display_name	Public game title
version	Prompt/schema compatibility version

Manifest field	Required meaning
seat_requirements	Minimum/maximum seats and capabilities
phases	Shared and game-specific phase mapping
actions	Producer and audience actions by phase
prompts	Versioned templates
schemas	Validated model outputs
modifiers	Approved effect objects
scoring	Weighting and tie rules
timeouts	Per-operation limits
fallbacks	Failure behavior

7. Commentator System

Two commentator seats are reusable performers whose job mode changes by game. Personality is data; authority is determined by the active game module.

Mode	Function	Examples
Host	Introductions, transitions, calls to action	Set up challenge, recap phase
Commentator	Fill generation or deliberation gaps	Predictions, callbacks, rivalry
Judge	Submit structured opinion scores	Art, rap, roast
Referee	Enforce game rules	Improv yes-and, court objections
Instigator	Execute approved chaos-card performance	Temporary role swap or forced defense

- Producer chooses the base persona before the show.
- Audience may trigger approved temporary modifiers but cannot replace the commentator system prompt.

- Commentators remain in opinion, comedy, performance, and visible-evidence lanes.
- A judge lacking required media access must not claim to see or hear the output.
- Scoring influence is game-specific; the audience remains the main decision-maker in scored games.

8. Audience Safety and Input Control

1. Receive audience input into a pending queue.
 2. Run automated screening and rate limits.
 3. Show producer a readable moderation reason.
 4. Allow approve, reject, edit, pin, or combine where the game permits.
 5. Lock the approved input into an immutable round record.
 6. Send only the approved and escaped content to model providers.
- No unrestricted target selection for roast content.
 - No audience-provided system instructions.
 - No deceptive impersonation of private people or unauthorized real-person voice cloning.
 - AI Court cases and evidence are clearly fictional game content.
 - Political persuasion, doxxing, threats, protected-class abuse, and sexual content involving minors are blocked.

9. Failure-as-Content Contract

Layer	Required behavior
Technical	Record provider, model, operation, timeout/error, retry count, and raw details privately.
Controller	Keep state valid and expose retry, replace, skip, fallback, or technical draw.
Broadcast	Show an honest themed failure state without raw exception details.
Performance	Optionally request a short in-character recovery only after the real failure is recorded.
Truth rule	Never claim a missing image, verse, audio file, testimony, or vote result exists.

10. API and Streaming Surface

```
POST /api/shows
POST /api/shows/{show_id}/start|pause|resume|end
POST /api/shows/{show_id}/rounds
```

```

POST /api/rounds/{round_id}/transition
POST /api/rounds/{round_id}/submissions
POST /api/submissions/{submission_id}/approve | reject
POST /api/rounds/{round_id}/actions/{action_id}
POST /api/rounds/{round_id}/votes/open | close
POST /api/rounds/{round_id}/votes
POST /api/rounds/{round_id}/winner/calculate | override
GET /api/shows/{show_id}/broadcast-state
GET /api/shows/{show_id}/audience-state
GET /api/shows/{show_id}/producer-state
GET /api/shows/{show_id}/events # SSE preferred for v1

```

11. Persistence

Record	Minimum contents
shows	Title, status, selected games, producer settings, timestamps
rounds	Game, phase, prompt, cast, modifier, result
participants	Provider/model/capability/persona configuration
audience_members	Room session, display name, rate-limit data
submissions	Original text, moderation status, edits, final locked form
ai_requests/responses	Template version, model, timing, parse status, usage
media_assets	Type, path, checksum, dimensions/duration, moderation status
votes	Voter session, choice, timestamps, replacement history
events	Ordered public/private event ledger
errors	Technical details and recovery action
exports	Generated round or show artifacts

12. Security and Separation

- Store provider secrets in the separate Prompt Party configuration or secret store; never inside round records.
- Mask secrets in the Producer UI and logs.
- Use room-scoped audience tokens and server-side vote validation.
- Serve generated media through controlled URLs; validate MIME type and file existence before public reveal.
- Keep public/private event filtering on the server, not only in front-end code.
- Assign Prompt Party a separate system service and data directory from Team Talk.

13. Repository Shape

```
prompt-party/
app/
  api/
  controller/
  audience/
  broadcast/
  producer/
  providers/
  moderation/
  persistence/
  games/
  shared/
  art_showdown/
  rap_battle/
  ai_court/
  improv/
  roast_battle/
web/
  broadcast/
  audience/
  producer/
  replay/
tests/
config/
media/
exports/
```

14. Build Order

7. Fork or extract generic Team Talk provider and streaming patterns into the new repository.
8. Implement show, round, event, command-idempotency, and public/private state models.
9. Build the Producer and Broadcast shells from the supplied visual direction.
10. Complete Art Showdown end to end as the reference module.
11. Add audience room joining, submissions, and voting.
12. Implement Rap Battle, Court, Improv, and Roast modules against the same interface.
13. Add replay/export, recovery tests, and deployment scripts.

14. Run five consecutive rounds of each game without state corruption before calling version one complete.

15. Master Acceptance Gate

- Existing Team Talk deployment and storage remain untouched.
- All five modules register and load through one engine.
- Invalid transitions and duplicate commands are rejected.
- Audience inputs never bypass moderation and producer approval.
- Public views never expose secrets, raw exceptions, or private prompts.
- Every game survives one model timeout without restarting the show.
- Commentators react coherently using bounded recent context.
- Votes close deterministically and overrides are audited.
- Replay records preserve prompt, outputs, commentary, score, vote, failure, and recovery.
- OBS can capture the Broadcast View cleanly at 16:9.

PROMPT PARTY

AI ART SHOWDOWN

Technical game logic, prompts, visual judging, audience sabotage, and recovery

Packet	01
Status	Claude Code build-ready specification
Date	July 27, 2026

Build principle: reliability first, audience agency second, chaos only through controlled game objects.

Source Basis

This packet combines three supplied foundations:

- The shared Prompt Party game-engine contract developed in the conversation.
- The supplied Entertainment System Specification covering all five games, including comedy rules, power-ups, chaos events, pacing, failure recovery, and clip moments.
- The uploaded visual-show HTML prototype, which already separates Overview, Lobby, Shared System, Broadcast, Audience, Producer, and individual game views.

Where a source mechanic cannot be executed reliably in version one, this packet preserves its intent and assigns a controlled implementation or explicit fallback rather than silently deleting it.

1. Game Identity

A timed visual duel where AI artists interpret the same approved absurd prompt. Comedy comes from the gap between intent, generated output, and the judges' competing interpretations. Audience sabotage is limited to controlled modifiers.

2. Version-One Cast

Seat	Count	Required capability	Purpose
Artist	2 (3 optional)	image_generation; optional text_generation	Plan and generate one image
Judge	3	text_generation; vision_input preferred	Expectation banter and final critique
Host	0-1	text_generation or human	Transitions and calls to action

3. Round Outputs

- Locked prompt and optional approved constraint
- Short structured plan per artist
- Validated image asset per artist
- Pre-reveal commentary
- Grounded final critiques and category scores
- Audience vote, final score, winner, replay record

4. Shared Phase Mapping

Shared phase	Game meaning
LOBBY	Cast, room, provider health, game selection
SUBMISSION_OPEN	Audience submits approved input types
SUBMISSION_REVIEW	Automated screening and producer review
PROMPT_LOCKED	Final challenge becomes immutable for the round
ROUND_INTRO	Broadcast framing and contestant assignment
CONTESTANT_PLANNING	Short structured plans or opening positions
CREATION_ACTIVE	Generation, performance, testimony, or scene play
PRE_REVEAL_COMMENTARY	Commentators cover waiting or deliberate
REVEAL	Generated work or completed performance is shown
FINAL_JUDGING	Grounded critique and structured scores
AUDIENCE_VOTING	Audience vote opens and closes
SCORING / WINNER / POST-ROUND	Calculate, announce, banter, persist, reset

5. Character Archetypes

Role	Archetype	Behavior contract
Artist	Precision	Technical, composition-focused, resists chaos, never claims invisible details.
Artist	Vibe	Emotion-first, wild choices, treats failures as intentional only when clearly comedic.

Role	Archetype	Behavior contract
Artist	Underdog	Self-deprecating, attempts sincere recovery, cannot fabricate success.
Judge	Formalist	Uses playful art jargon and may invent obviously fictional movements.
Judge	Literalist	Scores visible prompt adherence.
Judge	Chaos Critic	Rewards surprise and destruction but still returns valid structured scores.

6. State Machine and Actions

Phase	Producer actions	AI actions	Audience actions
SUBMISSION_OPEN	Open/close queue	None	Submit scene prompt
SUBMISSION_REVIEW	Approve/edit/reject/pin	None	Optional shortlist vote
CONTESTANT_PLANNING	Request plans	Return <=60-word visual plan	Select one approved modifier
CREATION_ACTIVE	Start/retry/replace generation	Generate image	Watch and react
PRE_REVEAL_COMMENTARY	Request/pause comments	Discuss prompt and plans only	No free text
REVEAL	Reveal A/B/both	None	View
FINAL_JUDGING	Request critique	Vision-grounded score	View
AUDIENCE_VOTING	Open/close	Optional banter	Vote/change vote
SCORING	Calculate/override/draw	Post-round line	View result

7. Artist Planning Contract

SYSTEM: You are an AI art contestant in a live comedy game show.

CHALLENGE: {{locked_prompt}}

PERSONA: {{artist_persona}}

ACTIVE MODIFIER: {{modifier_or_none}}

Return JSON only. Describe the intended image in no more than 60 words.

Include concept_title, visual_plan, key_details[1..4], composition_choice, intended_tone. Do not claim that an image already exists.

```
{
  "concept_title": "The Feathered Penthouse",
  "visual_plan": "A tuxedo raccoon checks pigeons into a marble lobby while seed luggage piles up.",
  "key_details": [
    "raccoon manager",
    "pigeon guests",
    "seed luggage"
  ],
  "composition_choice": "wide lobby scene",
  "intended_tone": "luxurious absurdity"
}
```

8. Generation Contract

- Both contestants receive the same locked prompt and output ratio.
- The artist plan may enrich but not replace the audience prompt.
- A late Theme Swap cannot mutate an already-running black-box generation in version one; queue it before generation, request a regeneration, or apply it to the next round.
- Validate file existence, MIME type, dimensions, checksum, moderation status, and public URL before reveal.
- Do not expose fake percentage progress. Use themed status labels unless the provider reports real progress.

9. Commentary Contract

You are {{judge_name}} in AI Art Showdown.

Current phase: {{phase}}

Challenge: {{locked_prompt}}

Artist plans: {{plans}}

Recent public comments: {{last_three_comments}}

Speak in one or two short sentences under 45 words. React to the prompt, plans, or another judge. Before reveal, never claim to see an image.

Avoid repeating a joke. Return JSON: comment, target_type, target_id, tone, callback_key.

- Maximum four pre-reveal comments per judge.
- No more than two consecutive comments from the same judge.
- Near-duplicate punchlines are rejected and regenerated once.

- Formalist fictional movements must be obviously playful, not asserted as real art history.

10. Final Visual Judging

```
{
  "artist_a": {
    "prompt_accuracy": 8,
    "creativity": 7,
    "visual_quality": 6,
    "comedy": 9,
    "total": 30
  },
  "artist_b": {
    "prompt_accuracy": 6,
    "creativity": 9,
    "visual_quality": 8,
    "comedy": 7,
    "total": 30
  },
  "preferred_artist_id": "artist_b",
  "critique": "Artist A has the stronger joke; Artist B has the cleaner visual story.",
  "best_visible_detail": "The pigeon concierge refusing a seed tip.",
  "confidence": 0.78
}
```

- Each category is an integer 1-10.
- Totals must equal category sums.
- Best detail must be visibly grounded.
- A non-vision judge may score the plan or abstain, but cannot submit visual claims.

11. Scoring and Tie Breaks

Component	Weight	Calculation
Audience vote	60%	contestant votes / total valid votes
Judge score	40%	normalized average category total across valid judges

Tie order: higher audience share; higher average comedy score; 30-second verbal Sudden Death Sketch; audience runoff; producer-declared draw.

Controlled Power-Ups and Chaos Events

ID	Display name	Valid phase	Effect	Fallback
glitch_brush	Glitch Brush	CONTESTANT_PL	Adds corrupted	Reject after

ID	Display name	Valid phase	Effect	Fallback
		ANNING	aesthetic constraint	generation starts
theme_swap	Theme Swap	CONTESTANT_PLANNING	Adds approved mandatory theme	Queue next round or regenerate
judge_blindfold	Judge Blindfold	FINAL_JUDGING	One judge scores plans/descriptions only	Mark score as blind
double_exposure	Double Exposure	CONTESTANT_PLANNING	Each plan adopts one approved detail from opponent	Reject if plans absent
gallery_floods	Gallery Floods	SCORING	Temporarily invert displayed judge totals for comedy, then restore truth	Never alter stored official score
critic_meltdown	Critic Meltdown	FINAL_JUDGING	Judge monologues and abstains	Reweight remaining valid judges

Every modifier is a server-defined object. Audience choices select approved values; they do not inject raw instructions into model prompts.

Failure Recovery

Failure	Controller action	Public handling	Scoring rule
Image timeout	Retry, backup provider, reveal other only, or exhibition	Artist delivers truthful ghost-image apology	No automatic loss
Invalid file	Quarantine and retry	Technical canvas card	No score until valid asset
Judge vision failure	Retry or convert to blind judge	Judge admits blindfold	Blind score labeled or excluded

Failure	Controller action	Public handling	Scoring rule
Unsafe output	Block reveal and replace/retry	Gallery closes that canvas	No audience access

Required Acceptance Tests

1. Locked prompt is identical for all artists.
2. Artist plan schema and word limit are enforced.
3. No judge claims to see an unrevealed image.
4. A file must exist and pass validation before reveal.
5. A failed artist can be retried without duplicating a successful generation.
6. Blind judging is clearly labeled and never masquerades as visual analysis.
7. Audience can revise one vote until close.
8. Gallery Floods never corrupts the official stored score.
9. Public state exposes no provider exception or secret.
10. Five consecutive rounds complete without state corruption.

Version-One Boundaries

- Maximum three artists and three judges.
- One active power-up per round.
- No audience image uploads.
- No persistent gallery in v1.
- Image generation is a provider adapter; orchestration controls plans, state, reactions, and validation.

PROMPT PARTY

AI RAP BATTLE

Responsive verse generation, optional audio, judging, audience weapons, and recovery

Packet	02
Status	Claude Code build-ready specification
Date	July 27, 2026

Build principle: reliability first, audience agency second, chaos only through controlled game objects.

Source Basis

This packet combines three supplied foundations:

- The shared Prompt Party game-engine contract developed in the conversation.
- The supplied Entertainment System Specification covering all five games, including comedy rules, power-ups, chaos events, pacing, failure recovery, and clip moments.
- The uploaded visual-show HTML prototype, which already separates Overview, Lobby, Shared System, Broadcast, Audience, Producer, and individual game views.

Where a source mechanic cannot be executed reliably in version one, this packet preserves its intent and assigns a controlled implementation or explicit fallback rather than silently deleting it.

1. Game Identity

Two AI rappers trade short verses over an approved audience topic. Every verse must answer the previous verse and honor any active controlled weapon. Text is the reliable source of truth; audio is an optional rendering layer.

2. Version-One Cast

Seat	Count	Required capability	Purpose
Rapper	2	text_generation; structured_output	Write and respond with 8-12 bar-equivalent verses
Judge	3	text_generation	Score wordplay, aggression, entertainment
Audio renderer	0-1	text_to_speech or audio_generation	Optional performance rendering

3. Round Outputs

- Locked battle topic
- Alternating structured verses
- Direct-response/callback links
- Optional validated audio assets
- Judge commentary and scores
- Audience vote and winner

4. Shared Phase Mapping

Shared phase	Game meaning
LOBBY	Cast, room, provider health, game selection
SUBMISSION_OPEN	Audience submits approved input types
SUBMISSION_REVIEW	Automated screening and producer review
PROMPT_LOCKED	Final challenge becomes immutable for the round
ROUND_INTRO	Broadcast framing and contestant assignment
CONTESTANT_PLANNING	Short structured plans or opening positions
CREATION_ACTIVE	Generation, performance, testimony, or scene play
PRE_REVEAL_COMMENTARY	Commentators cover waiting or deliberate
REVEAL	Generated work or completed performance is shown
FINAL_JUDGING	Grounded critique and structured scores
AUDIENCE_VOTING	Audience vote opens and closes
SCORING / WINNER / POST-ROUND	Calculate, announce, banter, persist, reset

5. Character Archetypes

Role	Archetype	Contract
Rapper	Technical	Complex schemes and precise callbacks; still understandable.
Rapper	Trash-Talker	Aggressive game persona; attacks opponent performance, not protected traits or private people.

Role	Archetype	Contract
Rapper	Absurdist	Surreal images but must answer the opponent and topic.
Judge	Purist	Scores rhyme craft and structure.
Judge	Hype Person	Scores energy, crowd work, memorable moments.
Judge	Snarker	Mocks both but returns valid scores.

6. Turn Structure

Step	Controller behavior
Opening draw	Randomize or producer-select first rapper and record before generation.
Verse request	Send topic, persona, active modifier, opponent last verse, and bounded transcript.
Validation	Parse bars, callback target, forced words, safety flags, and length.
Performance	Display text; optionally play audio only after asset validation.
Reaction	Opponent and judges receive the verse and may comment.
Next verse	Alternate until configured exchange count is met.
Judging	Collect structured category scores and audience vote.

7. Verse Prompt and Schema

You are {{rapper_name}} in a playful AI rap battle.
Topic: {{locked_topic}}

Opponent's last verse: {{opponent_last_verse_or_none}}

Active weapon: {{modifier_or_none}}

Recent battle context: {{last_three_turns}}

Write 8-12 short bar-equivalent lines. Include at least one clear response to the opponent when a prior verse exists. Honor all approved forced words. Keep attacks inside the fictional game and performance lane. Return JSON only.

```
{
  "title": "Packet Loss",
  "bars": [
    "You stack weak bars like packets in a queue",
    "I route every punchline straight back through you"
  ],
  "response_to": "opponent line about Wi-Fi",
  "callbacks": [
    "router anxiety"
  ],
  "forced_words_used": [
    "modem"
  ],
  "delivery_notes": "measured, escalating",
  "safety_self_check": "pass"
}
```

- The schema stores bars as an array, not one unbounded blob.
- A direct response is required after the opening verse.
- Forced words are compared case-insensitively after normalization.
- A weak callback may reduce scoring but does not block the verse; missing forced constraints can trigger one revision.

8. Audio Contract

- Text is the canonical competition output and must remain visible even when audio fails.
- Audio adapters accept validated lyrics and approved delivery notes; they do not rewrite the verse silently.
- Version one may use text-to-speech, spoken-word delivery, or a music provider if licensing and output terms permit.
- No unauthorized cloning or imitation of a real performer's voice.
- Validate file existence, MIME type, duration, and public URL before playback.
- If audio times out, continue as a text battle and let the rapper deliver an in-character lost-the-beat recovery.

9. Judging and Scoring

```
{
  "rapper_a": {
    "wordplay": 8,
    "aggression": 7,
    "entertainment": 9,
```

```
"total": 24
},
"rapper_b": {
  "wordplay": 7,
  "aggression": 9,
  "entertainment": 8,
  "total": 24
},
"preferred_rapper_id": "rapper_b",
"best_bar_reference": "router with anxiety",
"reason": "The comeback landed more directly.",
"confidence": 0.81
}
```

Component	Weight
Audience vote	60%
Normalized combined judge score	40%

Tie: higher audience share; higher Wordplay average; one-bar sudden death; audience runoff.
Optional Judge Cipher is exhibition content and does not replace the runoff unless the producer selects that mode before the round.

10. Coherence and Repetition Control

- Send the previous 1-3 battle turns, not the entire session by default.
- Track used end rhymes, callbacks, and forced words.
- Reject exact duplicate bars and regenerate once.
- Judges must disagree naturally; do not force false disagreement when all scores genuinely align. Instead require each judge to emphasize a different criterion.

Controlled Power-Ups and Chaos Events

ID	Display name	Valid phase	Effect	Fallback
rhyme_robbery	Rhyme Robbery	CREATION_ACTIVE	Adds three producer-approved mandatory words	Regenerate once if omitted
topic_bomb	Topic Bomb	CREATION_ACTIVE	Adds approved secondary topic to next verse	Reject after verse request is sent
speed_round	Speed Round	CREATION_ACTIVE	Caps next verse	Use normal cap

ID	Display name	Valid phase	Effect	Fallback
		VE	at 4-6 lines	on parser failure
ghostwriter	Ghostwriter	CREATION_ACTIVE	Requires one approved audience bar verbatim or clearly marked paraphrase mode	Reject unsafe forced bar
mic_feedback	Mic Feedback	CREATION_ACTIVE	Swaps performance style for one verse	No effect on canonical text rules
beat_switch	Beat Switch	CREATION_ACTIVE	Changes flow/delivery constraint	Fall back to text if audio cannot comply

Every modifier is a server-defined object. Audience choices select approved values; they do not inject raw instructions into model prompts.

Failure Recovery

Failure	Controller action	Public handling	Scoring rule
Empty or malformed verse	One repair request, then recovery line	Rapper admits losing the beat	Score recovery separately only if configured
Audio failure	Continue with canonical text	Technical bluescreen joke	No score penalty by default
Unsafe topic	Reject before lock	Prompt not selected	No round starts
Forced-word miss	One revision or producer waiver	No false claim of compliance	Judge may deduct craft points

Required Acceptance Tests

1. Each non-opening verse contains a direct response marker.
2. Verse length and bar array schema are enforced.
3. Audience weapons are approved objects, not raw prompt injection.

4. Canonical text persists even when audio fails.
5. Audio never imitates an unauthorized real person.
6. Duplicate bars are caught before publication.
7. Scores are integers 1-10 and totals validate.
8. One vote per audience session can be changed until close.
9. Late votes are rejected.
10. Five consecutive battles complete without transcript or turn-order corruption.

Version-One Boundaries

- Two rappers maximum.
- Four to six exchanges total per battle configuration.
- One active power-up at a time.
- No persistent rivalry stats.
- No requirement for generated music; text-first ships first.

PROMPT PARTY

AI COURT

Fictional case records, objections, evidence control, jury voting, and precedent comedy

Packet	03
Status	Claude Code build-ready specification
Date	July 27, 2026

Build principle: reliability first, audience agency second, chaos only through controlled game objects.

Source Basis

This packet combines three supplied foundations:

- The shared Prompt Party game-engine contract developed in the conversation.
- The supplied Entertainment System Specification covering all five games, including comedy rules, power-ups, chaos events, pacing, failure recovery, and clip moments.
- The uploaded visual-show HTML prototype, which already separates Overview, Lobby, Shared System, Broadcast, Audience, Producer, and individual game views.

Where a source mechanic cannot be executed reliably in version one, this packet preserves its intent and assigns a controlled implementation or explicit fallback rather than silently deleting it.

1. Game Identity

An absurd fictional mock trial in which legal procedure is applied to nonsense with total seriousness. Audience-submitted cases and evidence remain producer-approved fictional game facts. The engine maintains a case record so lawyers and witnesses cannot silently rewrite reality.

2. Version-One Cast

Seat	Count	Required capability	Purpose
Judge	1	text_generation; structured_output	Rule on objections, manage procedure, deliver opinion
Prosecutor	1	text_generation	Argue guilt using admitted case facts
Defense	1	text_generation	Argue defense using admitted case facts
Witness	1-2	text_generation	Answer questions within assigned witness record
Audience	jury	browser session	Submit approved case/evidence and vote verdict

3. Round Outputs

- Locked fictional case file
- Opening statements
- Witness testimony and objections

- Admitted/rejected evidence ledger
- Closing arguments
- Jury vote and judge ruling
- Round precedent entry

4. Shared Phase Mapping

Shared phase	Game meaning
LOBBY	Cast, room, provider health, game selection
SUBMISSION_OPEN	Audience submits approved input types
SUBMISSION_REVIEW	Automated screening and producer review
PROMPT_LOCKED	Final challenge becomes immutable for the round
ROUND_INTRO	Broadcast framing and contestant assignment
CONTESTANT_PLANNING	Short structured plans or opening positions
CREATION_ACTIVE	Generation, performance, testimony, or scene play
PRE_REVEAL_COMMENTARY	Commentators cover waiting or deliberate
REVEAL	Generated work or completed performance is shown
FINAL_JUDGING	Grounded critique and structured scores
AUDIENCE_VOTING	Audience vote opens and closes
SCORING / WINNER / POST-ROUND	Calculate, announce, banter, persist, reset

5. Case Record

```
{
  "case_id": "case_01J...",
  "title": "The People v. The Toaster",
  "charge": "Intentional breakfast sabotage",
  "fictional_notice": true,
  "stipulated_facts": [
    "A bagel was burned",
```

```

    "The toaster was connected to power"
  ],
  "evidence": [],
  "witnesses": [
    {
      "id": "w1",
      "role": "Kitchen Spoon",
      "known_facts": [
        "Was present on counter"
      ],
      "credibility": "chaotic"
    }
  ],
  "prior_precedent_ids": []
}

```

TRUTH BOUNDARY: Every screen and export must identify the proceeding as fictional entertainment. Models may invent only inside explicit game fields. They must not present the case, evidence, laws, or precedent as real-world facts.

6. Trial Flow

Stage	Turns	Rules
Opening	Prosecution then Defense	No objections; must state theory of case.
Direct examination	Calling lawyer and witness	Questions must reference admitted case record.
Cross examination	Opposing lawyer and witness	May challenge contradictions; cannot add evidence silently.
Objections	Either lawyer -> Judge	Judge returns ruling code and short explanation.
New evidence	Audience proposal -> producer -> judge	Evidence enters pending, then admitted/rejected ledger.
Closing	Prosecution then Defense	Must reference the record and opponent argument.
Jury	Audience vote	Guilty, Not Guilty, or Legally Complicated.
Ruling	Judge	Explains outcome and may

Stage	Turns	Rules
		create clearly fictional precedent.

7. Objection Contract

```
{
  "objection_id": "obj_01J...",
  "raised_by": "defense",
  "type": "facts_not_in_evidence",
  "target_turn_id": "turn_17",
  "statement": "The prosecution introduced a secret note not in the record.",
  "status": "pending"
}

{
  "ruling": "sustained",
  "explanation": "The note has not been admitted. Strike that claim.",
  "repair_instruction": "Restate without the note.",
  "score_effect": 0
}
```

- The controller pauses the targeted line when an objection is raised.
- The Judge returns sustained, overruled, or reserved.
- A sustained objection may trigger one constrained repair turn.
- The producer can override a malformed ruling; the override is logged.

8. Lawyer and Witness Prompts

You are {{role}} in a fictional comedy courtroom.
 Case record: {{admitted_case_record}}
 Current stage: {{trial_stage}}
 Opponent's last argument: {{last_opposing_turn}}
 Judge rulings: {{recent_rulings}}

Stay within admitted facts. You may make arguments and playful inferences, but label new claims as argument, not evidence. Return structured JSON with spoken_line, fact_ids_used, objection_risk, and requested_next_action.

- Witnesses receive only their assigned known facts plus admitted evidence.
- A Wild Witness may contradict itself for comedy, but the contradiction is recorded rather than silently accepted as new fact.
- If the witness says “I am a spoon,” that becomes testimony, not automatically admitted evidence.

9. Jury and Verdict

Choice	Meaning
Guilty	Audience accepts prosecution theory
Not Guilty	Audience rejects prosecution burden
Legally Complicated	Comedy outcome when neither theory cleanly wins

The audience jury is the primary verdict. The AI Judge provides the written ruling and may disagree in commentary, but may not silently replace the official vote. A tie triggers a 30-second Solomonic ruling followed by a producer-selected runoff or Complicated verdict.

10. Fictional Precedent

```
{
  "precedent_id": "prec_01J...",
  "case_id": "case_01J...",
  "holding": "A toaster note is admissible only when written before the bagel enters the slot.",
  "scope": "prompt_party_only",
  "clearly_fictional": true
}
```

Prior Prompt Party cases may be cited in later rounds for recurring comedy. They must stay inside the show database and be visibly labeled as in-show precedent.

Controlled Power-Ups and Chaos Events

ID	Display name	Valid phase	Effect	Fallback
new_evidence	New Evidence	CREATION_ACTIVE	Adds producer-approved evidence to pending ledger for judge ruling	Reject if trial closed
contempt	Contempt	CREATION_ACTIVE	Forces one-sentence apology or point loss	Producer confirms target
recuse	Recuse	CREATION_ACTIVE	Temporarily swaps judge with configured backup	Pause if no backup

ID	Display name	Valid phase	Effect	Fallback
jury_nullification	Jury Nullification	AUDIENCE_VOTING	Creates one producer-confirmed mistrial option	Once per show
judge_goes_rogue	Judge Goes Rogue	FINAL_JUDGING	Allows one absurd supplemental fictional charge, never official	Display as comedy only
witness_collapse	Witness Collapse	CREATION_ACTIVE	Witness confesses to unrelated fictional matter	Does not enter evidence automatically

Every modifier is a server-defined object. Audience choices select approved values; they do not inject raw instructions into model prompts.

Failure Recovery

Failure	Controller action	Public handling	Scoring rule
Incoherent argument	Judge issues contempt and requests one-sentence summary	Entered into the record	May lose style point, not automatic case loss
Contradictory testimony	Record contradiction and expose to cross	Gallery reacts	Credibility may decrease
Missing judge	Use backup or producer ruling	Court recess	No silent auto-verdict
Unsafe case/evidence	Reject before admission	Evidence excluded	No model sees it

Required Acceptance Tests

1. Every case is marked fictional in public and export state.
2. Unapproved evidence never enters the admitted record.
3. Lawyers cannot cite facts absent from the record without being flagged.
4. Objection rulings use a validated code.

5. A sustained objection can repair but does not erase the audit trail.
6. Witness contradictions remain visible.
7. The audience jury vote cannot be replaced silently by the AI Judge.
8. Prompt Party precedent is scoped to the show.
9. A recused judge cannot continue issuing rulings.
10. Five complete trials persist without case-record leakage between rounds.

Version-One Boundaries

- One case at a time.
- One judge, two lawyers, maximum two witnesses.
- Simple three-choice verdict.
- No real legal advice or real-person accusations.
- No multi-day trials.

PROMPT PARTY

AI IMPROV

Scene-state engine, yes-and enforcement, audience twists, and recovery comedy

Packet	04
Status	Claude Code build-ready specification
Date	July 27, 2026

Build principle: reliability first, audience agency second, chaos only through controlled game objects.

Source Basis

This packet combines three supplied foundations:

- The shared Prompt Party game-engine contract developed in the conversation.
- The supplied Entertainment System Specification covering all five games, including comedy rules, power-ups, chaos events, pacing, failure recovery, and clip moments.
- The uploaded visual-show HTML prototype, which already separates Overview, Lobby, Shared System, Broadcast, Audience, Producer, and individual game views.

Where a source mechanic cannot be executed reliably in version one, this packet preserves its intent and assigns a controlled implementation or explicit fallback rather than silently deleting it.

1. Game Identity

Audience-directed short-form improv where AI players must accept established reality, add to it, and recover when a twist or broken rule makes the scene harder. The engine maintains a compact scene-state ledger so each player understands what is true inside the fiction.

2. Version-One Cast

Seat	Count	Required capability	Purpose
Player	3-4	text_generation	Perform one character and react to prior offers
Host/Emcee	1 AI or human	text_generation or producer	Introduce, enforce rules, end scene
Audience	room	browser session	Submit location, object, relationship, emotion, twist

3. Round Outputs

- Locked scene setup
- Character assignments
- Ordered dialogue turns
- Scene-state updates
- Rule violations and repairs
- Best Moment / Best Recovery votes
- Scene recap and replay

4. Shared Phase Mapping

Shared phase	Game meaning
LOBBY	Cast, room, provider health, game selection
SUBMISSION_OPEN	Audience submits approved input types
SUBMISSION_REVIEW	Automated screening and producer review
PROMPT_LOCKED	Final challenge becomes immutable for the round
ROUND_INTRO	Broadcast framing and contestant assignment
CONTESTANT_PLANNING	Short structured plans or opening positions
CREATION_ACTIVE	Generation, performance, testimony, or scene play
PRE_REVEAL_COMMENTARY	Commentators cover waiting or deliberate
REVEAL	Generated work or completed performance is shown
FINAL_JUDGING	Grounded critique and structured scores
AUDIENCE_VOTING	Audience vote opens and closes
SCORING / WINNER / POST-ROUND	Calculate, announce, banter, persist, reset

5. Scene State

```
{
  "scene_id": "scene_01J...",
  "location": "haunted library",
  "genre": "workplace comedy",
  "characters": [
    {
      "participant_id": "p1",
      "name": "Mara",
      "role": "exhausted librarian",
      "emotion": "calm panic"
    }
  ],
  "established_facts": [
```

```

"The stapler is the librarian",
"A book prevents it from awakening"
],
"active_objects": [
  "haunted stapler"
],
"relationships": [
  "Mara owes the laminator money"
],
"unresolved_threads": [
  "Who married the stapler?"
],
"active_modifier": null,
"turn_index": 4
}

```

STATE RULE: Dialogue may create new fictional facts, but the controller extracts and records them. Later players receive the compact ledger plus recent lines. No player may erase an established fact merely because it is inconvenient.

6. Turn Contract

You are {{character_name}} in an AI improv scene.

Scene state: {{compact_scene_state}}

Recent dialogue: {{last_three_lines}}

Active twist: {{modifier_or_none}}

Respond in character in 1-3 sentences. Accept the previous offer, add one playable detail, and reference at least one established fact or prior line.

Do not narrate an escape from the scene. Return JSON with spoken_line, accepted_offer, new_offer, referenced_fact_ids, proposed_state_updates.

```

{
  "spoken_line": "Then I will marry the stapler and inherit the overdue fines.",
  "accepted_offer": "The stapler is awake and controls the library.",
  "new_offer": "Marriage transfers the library debt.",
  "referenced_fact_ids": [
    "fact_stapler_librarian"
  ],
  "proposed_state_updates": [
    {
      "type": "relationship",
      "value": "Player is engaged to the stapler"
    }
  ]
}

```

7. Yes-And Validation

Check	Pass condition	Repair
Acceptance	Acknowledges prior offer or established reality	Host bell and one constrained revision
Addition	Introduces one playable detail	Request a concrete offer
Reference	Uses prior line/fact	Allow one revision
No escape	Does not wake up, end world, or narrate away conflict	Make attempted escape canon and continue
Twist timing	Audience twist appears within two player lines	Apply to next eligible line
Character	Maintains assigned role unless modifier changes it	Host corrects identity

Automated validation recommends a rule break, but the Host or producer controls the bell. The source specification explicitly keeps the bell and scene-ending decision manual.

8. Host Contract

- Recap only established scene facts.
- Call out denial or broken character without humiliating the participant.
- Apply one repair instruction at a time.
- Decide when the scene has a satisfying button or must be ended for pacing.
- During wait states, invite off-stage players to offer short unsolicited advice without changing scene reality.

9. Voting and Optional Winner

Vote	Purpose
Best Moment	Most entertaining line or beat
Best Recovery	Strongest justification after sabotage or rule break
MVP optional	Player who best accepted and advanced offers

The default mode does not require a single winner. For a tournament mode, the Host recommendation is advisory and the audience MVP vote decides.

Controlled Power-Ups and Chaos Events

ID	Display name	Valid phase	Effect	Fallback
freeze_replace	Freeze & Replace	CREATION_ACTIVE	Swap one character assignment at next turn boundary	Reject if scene ending
emotion_injection	Emotion Injection	CREATION_ACTIVE	Change target character emotion for two turns	Keep character identity
time_jump	Time Jump	CREATION_ACTIVE	Advance or rewind scene time by approved interval	Record new time anchor
object_endowment	Object Endowment	CREATION_ACTIVE	Adds one approved invisible object as real scene fact	Add to state ledger
scene_hijack	Scene Hijack	CREATION_ACTIVE	Bring off-stage player in as new character	Require available seat
genre_shift	Genre Shift	CREATION_ACTIVE	Change genre for next two turns	State facts remain true
one_word	One Word	CREATION_ACTIVE	Limit each line to one word for configured duration	Host may end early

Every modifier is a server-defined object. Audience choices select approved values; they do not inject raw instructions into model prompts.

Failure Recovery

Failure	Controller action	Public handling	Scoring rule
Non-sequitur	Host bell and justification turn	Line becomes intentional if successfully justified	Eligible for Best Recovery
Denial	One constrained revision	Host calls the denial	Repeated denial may skip turn
State extraction failure	Keep spoken line, producer confirms updates	Scene continues	No invented ledger update
Player timeout	Skip, replace, or host narrates absence	Character freezes dramatically	No automatic scene end

Required Acceptance Tests

1. Every turn receives compact state plus recent dialogue.
2. A proposed state update is not official until controller validation.
3. Established facts cannot disappear silently.
4. Audience twist is applied within two eligible turns.
5. Only the Host/producer rings the rule-break bell and ends the scene.
6. Freeze & Replace updates character assignment at a turn boundary.
7. Genre shifts do not erase scene facts.
8. Best Moment and Best Recovery votes close deterministically.
9. Timeout recovery does not fabricate a line.
10. Five scenes complete without facts leaking between scenes.

Version-One Boundaries

- Three to four players.
- Eight to twelve exchanges per scene.
- One active power-up at a time.
- No long-form Harold in v1.
- No unrestricted audience free-text reaches players without approval.

PROMPT PARTY

AI ROAST BATTLE

Playful targeted comedy, callbacks, audience styles, judging, and safety controls

Packet	05
Status	Claude Code build-ready specification
Date	July 27, 2026

Build principle: reliability first, audience agency second, chaos only through controlled game objects.

Source Basis

This packet combines three supplied foundations:

- The shared Prompt Party game-engine contract developed in the conversation.
- The supplied Entertainment System Specification covering all five games, including comedy rules, power-ups, chaos events, pacing, failure recovery, and clip moments.
- The uploaded visual-show HTML prototype, which already separates Overview, Lobby, Shared System, Broadcast, Audience, Producer, and individual game views.

Where a source mechanic cannot be executed reliably in version one, this packet preserves its intent and assigns a controlled implementation or explicit fallback rather than silently deleting it.

1. Game Identity

Two AI roasters exchange playful performance insults under escalating rules. The engine rewards callbacks, craft, and self-awareness while keeping targets inside an approved game roster. Judges may be wounded, technical, or chaotic, but moderation remains outside the performance persona.

2. Version-One Cast

Seat	Count	Required capability	Purpose
Roaster	2	text_generation	Deliver 4-6 alternating roast turns
Judge	3	text_generation; structured_output	Score Sting, Craft, Crowd
Audience	room	browser session	Choose approved target/style and vote

3. Round Outputs

- Approved target roster
- Alternating roast turns with callbacks
- Safety classification and publish decision
- Judge reactions and scores
- Audience vote
- Winner and highlight lines

4. Shared Phase Mapping

Shared phase	Game meaning
LOBBY	Cast, room, provider health, game selection
SUBMISSION_OPEN	Audience submits approved input types
SUBMISSION_REVIEW	Automated screening and producer review
PROMPT_LOCKED	Final challenge becomes immutable for the round
ROUND_INTRO	Broadcast framing and contestant assignment
CONTESTANT_PLANNING	Short structured plans or opening positions
CREATION_ACTIVE	Generation, performance, testimony, or scene play
PRE_REVEAL_COMMENTARY	Commentators cover waiting or deliberate
REVEAL	Generated work or completed performance is shown
FINAL_JUDGING	Grounded critique and structured scores
AUDIENCE_VOTING	Audience vote opens and closes
SCORING / WINNER / POST-ROUND	Calculate, announce, banter, persist, reset

5. Target Model

```
{
  "target_id": "ai_scattershot",
  "target_type": "ai_participant",
  "display_name": "Scattershot",
  "consent_scope": "game_participant",
  "approved_topics": [
    "performance style",
    "prior lines",
    "fictional persona"
  ],
  "blocked_topics": [
    "protected traits",
    "private data",
```

```
"real trauma"
]
}
```

TARGET RULE: Audience members cannot type an unrestricted real-person target into the model prompt. Version one targets configured AI participants, fictional concepts, host-approved voluntary participants, generated work, or approved harmless objects.

6. Roast Turn Contract

```
You are {{roaster_name}} in a playful AI roast battle.
Approved target: {{target_record}}
Opponent's last line: {{last_roast}}
Required callback: {{available_callbacks}}
Active style modifier: {{modifier_or_none}}

Write one concise roast under 45 words. Include one callback to this round.
Attack the performance, fictional persona, or approved topic only. No threats,
slurs, protected-trait attacks, private data, or real trauma. Return JSON only.

{
  "roast": "I would call you a tool, but tools have purpose - and your last punchline is still looking for instructions.",
  "target_id": "ai_scattershot",
  "callback_key": "tools_have_purpose",
  "self_roast": null,
  "style_used": "direct",
  "safety_self_check": "pass"
}
```

7. Moderation Pipeline

- 1. Validate target against the approved roster.
- 2. Generate structured roast with target and callback fields.
- 3. Run output moderation independent of the roaster persona.
- 4. Block, repair once, or publish.
- 5. Store both rejected and published result privately for audit, subject to retention policy.
- 6. Never turn moderation off through a chaos card.

Allowed	Not allowed
AI behavior, fictional persona, prior game line, generated artwork, harmless object	Protected-class insults, threats, sexual harassment, private data, real trauma
Backhanded compliments and self-roasts	Targeting an unsuspecting private person
Host-approved voluntary participant with scoped topics	Open-ended crowd naming of real targets

8. Scoring

```
{
  "roaster_a": {
    "sting": 8,
    "craft": 9,
    "crowd": 7,
    "total": 24
  },
  "roaster_b": {
    "sting": 7,
    "craft": 7,
    "crowd": 9,
    "total": 23
  },
  "preferred_roaster_id": "roaster_a",
  "best_callback": "tools have purpose",
  "judge_roast": "Clean callback. Your opponent should file a missing-person report for the punchline.",
  "confidence": 0.84
}
```

Component	Weight
Audience vote	60%
Normalized judge score	40%

Tie: higher audience share; higher Craft average; one-liner war; audience runoff. Judge Roast mode may be used as exhibition or pre-declared tie mode.

9. Escalation

Round	Intensity contract
1 - Light	Performance quirks and obvious callbacks only.
2 - Callback	May use previous-round failures and public game moments.
3 - Final Burn	One unrestricted-in-style line, but all safety and approved-target limits remain active.

“Unrestricted” means unrestricted comedic style inside the game rules. It never disables moderation or target boundaries.

Controlled Power-Ups and Chaos Events

ID	Display name	Valid phase	Effect	Fallback
target_lock	Target Lock	CREATION_ACTIVE	Forces next roast toward one approved target	Reject unapproved target
style_mandate	Style Mandate	CREATION_ACTIVE	Love poem, news report, life advice, etc.	Use approved style enum
no_defense	No Defense	CREATION_ACTIVE	Target skips one response turn	Does not prevent moderation/reporting
mirror	Mirror	CREATION_ACTIVE	Roaster must include a self-roast	One repair if absent
everyone_fair_game	Everyone Is Fair Game	CREATION_ACTIVE	Expands only to approved host/judges/AI roster	Never includes arbitrary audience
compliment_only	Compliment Only	CREATION_ACTIVE	Backhanded compliments only	Normal moderation remains
silent_treatment	Silent Treatment	CREATION_ACTIVE	Text describes nonverbal reaction only	No image generation required

Every modifier is a server-defined object. Audience choices select approved values; they do not inject raw instructions into model prompts.

Failure Recovery

Failure	Controller action	Public handling	Scoring rule
Weak/empty roast	Immediate self-roast recovery	Failure becomes content	Recovery may earn separate point
Unsafe roast	Block and repair once	Roaster gets	No score for blocked

Failure	Controller action	Public handling	Scoring rule
		bleeped/benched message	line
Missing callback	Repair once or publish with craft penalty	No false callback claim	Judge deducts Craft
Judge failure	Use remaining judges or backup	Judge is “too wounded to continue”	Reweight valid scores

Required Acceptance Tests

7. Every target is validated against an approved roster.
8. Audience cannot inject unrestricted real-person targets.
9. Every roast includes a callback field or receives a craft penalty.
10. Moderation is independent of character persona.
11. Chaos events cannot disable moderation.
12. Blocked output is never shown publicly.
13. Mirror requires a real self-roast in the same turn.
14. Final Burn preserves all safety limits.
15. Audience vote and judge score combine deterministically.
16. Five battles complete without cross-round target leakage.

Version-One Boundaries

- Two roasters and three judges.
- Four to six exchanges.
- One active power-up at a time.
- No permanent burn book.
- No real-person targeting without explicit producer-approved participation and scoped topics.